

RESUMEN PROGRAMACION III. UNIDAD 5-7

5. Búsqueda Local

Motivación y Conceptos Generales

A diferencia de los algoritmos de búsqueda sistemática que vimos previamente (como BFS o A*) que consumen mucha memoria al mantener un árbol de caminos explorados, **una búsqueda local mantiene solo el estado actual (o unos pocos) en memoria**. En cada iteración, el algoritmo **evalúa y actualiza el estado actual moviéndose a uno de sus sucesores**, sin registrar el camino recorrido.

- **Aplicación:** Son ideales para **problemas de optimización**, donde lo único que importa es encontrar el **estado objetivo** y el camino para llegar a él es irrelevante.
- **Requisitos:** Requieren una **formulación de estados completa** (donde todos los elementos ya están en el tablero o asignados, y las acciones solo los modifican) y una **función objetivo** matemática que evalúe la calidad del estado, buscando alcanzar su máximo (o mínimo) valor.

Paisaje del Espacio de Estados

Para entender la búsqueda local, se visualiza el problema como un **paisaje topográfico**. La posición está dada por el estado y la "elevación" por el valor de la función objetivo. El algoritmo "camina" por este paisaje con el objetivo de **encontrar el valle más bajo (mínimo global) si se minimizan costos, o el pico más alto (máximo global) si se maximiza la función objetivo**. Un algoritmo óptimo siempre encontrará este extremo global.

Búsqueda de Ascensión de Colinas (Hill-Climbing Search)

Es la **búsqueda local más simple**, también conocida como **búsqueda local avara (greedy)** porque no mira más allá de sus sucesores inmediatos. En cada iteración, el algoritmo **se mueve siempre al sucesor con mayor valor objetivo**, de manera estrictamente ascendente.

Características y Performance:

- Tiene un **muy bajo consumo de memoria**.

- **Nunca cicla**, ya que siempre exige moverse a un estado con valor estrictamente mayor.
- Su gran defecto es que **se atasca frecuentemente en cimas que no son la óptima**, por lo que **no garantiza completitud ni optimalidad**.

Problemas del Paisaje (Por qué se atasca)

El éxito de este algoritmo depende fuertemente de la forma del paisaje, pudiendo quedar atrapado en los siguientes accidentes topográficos:

1. **Máximos locales:** Un pico que es más alto que todos sus vecinos inmediatos, pero más bajo que el máximo global.
2. **Crestas:** Una secuencia de máximos locales conectada, que obliga al algoritmo a realizar un zigzag ineficiente sin salida clara.
3. **Mesetas:** Un área donde la función de evaluación es completamente plana (los sucesores tienen el mismo valor objetivo que el estado actual). Se dividen en:
 - **Máximo local plano:** No existe ninguna salida ascendente.
 - **Terraza:** Existe la posibilidad de seguir ascendiendo si se logra cruzarla.

Movimientos Laterales

Para poder escapar de una meseta, **es posible permitir movimientos laterales**, es decir, moverse a un estado sucesor que tenga el **mismo valor objetivo** que el estado actual. Sin embargo, **permitir infinitos movimientos laterales no garantiza escapar y genera el riesgo de caer en un bucle infinito** (ciclar) si se está en un máximo local plano. Por ello, para garantizar la terminación del algoritmo, **se debe limitar el número de movimientos laterales consecutivos** mediante un parámetro.

Otras Variantes de la Ascensión de Colinas

Para mejorar la efectividad, se crearon variantes fundamentales:

1. **Ascensión de colinas estocástica:** En lugar del mejor absoluto, **elige el sucesor de forma aleatoria** entre todos aquellos que mejoren el valor objetivo actual.
2. **Ascensión de colinas de primera opción:** Genera estados sucesores de a uno al azar y **elige el primero que encuentre** que sea mejor que el estado actual (muy útil cuando hay miles de sucesores).

3. **Ascensión de colinas con reinicio aleatorio:** Realiza múltiples búsquedas; cuando se atasca, reinicia la búsqueda desde un estado inicial generado aleatoriamente. Este método es extremadamente poderoso y **garantiza optimalidad con cierta probabilidad**, resolviendo problemas inmensos (como millones de reinas) en fracciones de tiempo.
-

Búsqueda Tabú (Tabu Search)

Es una búsqueda local avanzada que mejora la ascensión de colinas al permitir **moverse siempre al sucesor con el mejor valor objetivo, sin importar si este es peor, igual o mejor** que el estado actual. Como esta flexibilidad introduce un grave riesgo de caer en bucles infinitos (ciclar subiendo y bajando), incorpora una **memoria de corto plazo conocida como Lista Tabú**.

La Lista Tabú

Su función es **restringir movimientos almacenando información de las últimas iteraciones** para prohibir temporalmente volver a estados recién visitados.

¿Qué elementos almacenar?

1. **Acciones:** Guarda las acciones que revertirían movimientos recientes (ej. si moví una pieza a la fila 2, prohíbo moverla de nuevo a su fila original).
2. **Estados:** Guarda los estados completos que fueron visitados recientemente. Restringe menos el camino, pero conlleva un **mayor consumo de tiempo y memoria**.
3. **Propiedades:** Guarda características o atributos específicos de los estados.

¿Por cuánto tiempo almacenarlos? Si se guardan los elementos para siempre, habría un **consumo de memoria excesivo** y una **mayor probabilidad de atascarse** al quedarse sin opciones. Por eso, existen dos enfoques para gestionar la lista:

- **Capacidad limitada:** La lista funciona como una **cola FIFO**; si se excede la capacidad, se borra el elemento más viejo.
- **Tenor de tabú:** Se filtran y borran los elementos que hayan permanecido en la lista **más iteraciones del límite establecido**.

Criterios de Parada y Seteo de Parámetros

Como la búsqueda tabú puede moverse a estados peores, debe guardar siempre en una variable aparte el "mejor estado histórico" encontrado. Además, dado que podría no terminar nunca, requiere **criterios de parada** estrictos:

1. Tras un número de **iteraciones totales** o tiempo de CPU.
2. Tras un número de **iteraciones sin mejoras** en el valor objetivo.
3. Cuando la función objetivo **sobrepasa un valor umbral**.

Dato clave: Este algoritmo es **extremadamente sensible al seteo de parámetros** (qué guardar, capacidad, tenor). La performance puede mejorar o empeorar drásticamente con un mínimo cambio, por lo que **se determinan de manera puramente experimental**.

Componentes Adicionales de Búsqueda Tabú

Para problemas difíciles, se le suman mecanismos extra de efectividad:

1. **Criterio de aspiración:** Como la lista tabú a veces es demasiado restrictiva y prohíbe acciones excelentes, este criterio **permite ignorar el estado tabú si la acción conduce a un valor objetivo que es mejor que el mejor estado encontrado hasta el momento** (no hay riesgo de ciclar porque rompe un récord histórico).
2. **Diversificación:** Mecanismo que usa una **memoria de largo plazo (memoria de frecuencia)** para forzar al algoritmo a explorar regiones inexploradas y no pasar todo su tiempo en un mismo sector. Tiene dos estrategias:
 - **De reinicio:** Reiniciar la búsqueda desde estados con componentes poco frecuentes.
 - **Continua:** Al elegir sucesores, **priorizar o premiar** aquellos con componentes poco frecuentes.

Búsqueda Local en Espacios Continuos

En el mundo real (como optimizar tiempos o coordenadas), las magnitudes son continuas, lo que significa que **los estados tienen infinitos sucesores**, imposibilitando el uso de los algoritmos discretos vistos arriba.

Para resolver soluciones en espacios continuos (como decidir las coordenadas óptimas para ubicar un aeropuerto), la búsqueda local abandona la revisión de

sucesos individuales y **utiliza el cálculo matemático mediante el Gradiente**. El gradiente en un punto evalúa la derivada de la función objetivo y **determina la dirección exacta de la subida más empinada**, realizando saltos directos sobre el mapa.

6. PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES (CSP)

Motivación y Definición

A diferencia de los algoritmos clásicos que tratan a los estados como "cajas negras", un **Problema de Satisfacción de Restricciones (CSP)** utiliza una representación estructurada para modelar **problemas de factibilidad**. El objetivo es hallar una asignación de valores que cumpla con todas las reglas del problema (como colorear un mapa o programar horarios), donde **el camino para llegar a la solución es irrelevante**,,,.

Un CSP se compone formalmente de tres elementos,:

1. **Variables ()**: Un conjunto de incógnitas .
2. **Dominios ()**: Un conjunto . Cada dominio es el "**menú**" de valores permitidos para la variable .
3. **Restricciones ()**: Un conjunto que especifica las **combinaciones de valores permitidas**. Cada restricción es un par , donde son las variables involucradas y es la relación matemática o simbólica de valores válidos,.

Soluciones y Asignaciones

El proceso de resolver un CSP implica generar **asignaciones** (dar valores a las variables). Se clasifican en,:

- **Por completitud**: Es **completa** si a cada variable del problema se le asignó un valor; es **parcial** si a algunas no.
- **Por legalidad**: Es **consistente (o legal)** si no viola absolutamente ninguna restricción del problema; es **inconsistente** si viola al menos una.
- **Solución de un CSP**: Es estrictamente una **asignación que sea al mismo tiempo COMPLETA y CONSISTENTE** (cero restricciones violadas),.

Clasificación de Variables y Restricciones

Según su Dominio (Variables),,:

- **Discreta finita:** Tienen un número específico de valores posibles (ej. colores, booleanos). El número de asignaciones posibles crece de forma exponencial: .
- **Discreta infinita:** Cadenas de números enteros (ej. tiempos de inicio en scheduling).
- **Continua:** Variables con números reales (ej. coordenadas exactas, telescopio Hubble).

Según las Variables Involucradas (Restricciones):

- **Unaria:** Restringe a una sola variable. **Se puede eliminar directamente** reduciendo el dominio de dicha variable de antemano.
- **Binaria:** Relaciona a exactamente dos variables (ej.).
- **Global:** Relaciona a un número arbitrario de variables (ej. restricción AllDiff donde múltiples variables deben tener valores distintos).

Observación fundamental: Toda restricción unaria, global o de orden alto **se puede transformar en un conjunto de restricciones puramente binarias**,. Por esto, los CSP se suelen modelar mediante un **Grafo de Restricciones**, donde los **nodos son variables** y los **arcos son restricciones binarias** entre ellas,.

Formulación para Búsqueda Clásica y Conmutatividad

Para aplicar algoritmos clásicos a los CSP, se formula de la siguiente manera,:

- **Estado inicial:** Asignación vacía {}.
- **Acciones:** Elegir una variable no asignada y darle un valor de su dominio que mantenga la asignación consistente.
- **Test objetivo:** ¿La asignación es completa?

La magia de la Conmutatividad: En los CSP, **el orden en que se asignan las variables no altera el resultado final**,. Gracias a esto, no necesitamos generar un árbol masivo evaluando todas las permutaciones de variables; **basta con asignar una única variable en cada nivel del árbol**, reduciendo drásticamente su tamaño y los caminos redundantes,.

Búsqueda Vuelta Atrás (Backtracking Search)

Es el algoritmo por defecto para resolver CSP. Se trata de una **Búsqueda Primero en Profundidad (DFS)** adaptada,.

- En cada nodo del árbol elige **una variable a la vez** y le prueba valores consistentes.
 - Utiliza una **implementación recursiva** altamente eficiente en memoria (no almacena una estructura de frontera explícita),.
 - **El Retroceso (Backtrack):** Si llega a una variable que **no tiene ningún valor legal disponible**, declara un fallo. Tras el fallo, la búsqueda **retrocede (vuelve atrás) a la decisión anterior** e intenta un camino alternativo,.
-

Backtracking Mejorado (Criterios Dinámicos)

Backtracking puro (ciego) es muy ineficiente. Para que funcione en problemas masivos, se aplican heurísticas de propósito general que deciden **qué variable elegir** y en **qué orden probar sus valores**,.

1. Ordenamiento de Variables (ELEGIR_VAR)

- **Mínimos Valores Restantes (MVR):** También conocida como la variable "más restringida" o "primero en fallar". **Elige la variable que tenga la menor cantidad de valores legales restantes en su dominio**,. Al forzar los fracasos rápido, poda inmensas ramas inútiles del árbol.
- **Grado Heurístico:** Se utiliza para desempatar en MVR. Se elige la variable que participa en la **mayor cantidad de restricciones** con el resto de variables no asignadas.

2. Ordenamiento de Valores (ORDENAR_DOM)

- **Valor Menos Restringido:** Una vez elegida una variable, **prefiere el valor que deje la mayor flexibilidad a sus vecinos**. Es decir, elige el valor que elimine la menor cantidad de opciones en los dominios de las variables relacionadas no asignadas.
-

Inferencia: Propagación de Restricciones

Podemos usar las restricciones para anticipar fallos y reducir los dominios dinámicamente antes de asignar.

1. Comprobación Hacia Adelante (Forward Checking)

Al asignar un valor a la variable , el algoritmo revisa todas las variables vecinas () no asignadas y **elimina de sus dominios cualquier valor que sea inconsistente con ,**.

- *Ventaja:* Reduce dominios tempranamente y detecta inconsistencias. Si el dominio de alguna variable queda vacío, se retrocede de inmediato.
- *Limitación:* Falla en mirar "lo suficientemente adelante" (no detecta conflictos entre dos variables futuras que aún no fueron asignadas).

2. Arco-consistencia (AC-3)

Es una forma de inferencia más potente. Se dice que una variable es arco-consistente con **si para TODO valor en el dominio de , existe al menos un valor compatible en el dominio de ,**.

- Si se elimina un valor, AC-3 **propaga esa restricción en cadena** a otros vecinos para asegurar que todo el grafo mantenga la consistencia.
- Logra podar muchísimos más valores que Forward Checking, aunque consume más tiempo de cálculo,.

Restricciones Globales Especiales

Para optimizar el procesamiento de restricciones que involucran muchas variables, existen algoritmos de propósito especial,:

- **Alldiff:** Requiere que todas las variables involucradas tengan valores distintos. *Test rápido de inconsistencia:* Si hay variables y juntas tienen solo posibles valores en sus dominios combinados, y , el problema es inconsistente y falla de inmediato,.
- **Atmost (Restricciones de recursos):** Limita la sumatoria de valores. *Test rápido:* Si la suma de los valores más pequeños posibles en el dominio de cada variable ya excede el límite máximo permitido (), es inconsistente.

Vuelta Atrás Inteligente: Salto-atrás (Backjumping)

La *vuelta atrás cronológica* estándar sube simplemente a la última variable asignada, lo que puede provocar repetición infinita de fallos inútiles.

- **Salto-atrás dirigido por conflicto (Conflict-directed backjumping):** Ante un callejón sin salida, el algoritmo identifica el **conjunto conflicto** (la lista de variables asignadas en el pasado que causaron directamente este fallo). En lugar de retroceder un solo paso, **salta hacia atrás directamente a la variable más reciente dentro de ese conjunto conflicto**, ignorando variables intermedias irrelevantes,.

Resolución mediante Búsqueda Local (Min-Conflicts)

Los CSP son problemas de optimización perfectos para algoritmos locales, los cuales operan modificando un estado completo en lugar de construir un árbol.

- **Formulación de estados completa:** El estado inicial ya tiene valores asignados a **todas** las variables (aunque viole reglas y sea inconsistente). Las acciones consisten en cambiarle el valor a una de ellas,.
- **Función objetivo:** Una fórmula matemática que cuenta **el número exacto de restricciones violadas**. El objetivo es minimizar esta función para que el valle topográfico llegue al mínimo global (la solución),.
- **Heurística de Mínimos-Conflictos:** En cada iteración, se elige al azar una variable que esté en conflicto. Luego, se le asigna el valor de su dominio que **minimice el número total de restricciones violadas** en todo el tablero,.
- **Performance:** Extremadamente eficaz. Puede resolver configuraciones monstruosas (como **un millón de reinas**) en un promedio de solo 50 pasos,. Para evitar atascarse en mesetas topográficas, se complementa con técnicas como movimientos laterales o listas tabú.

Estructura de los Problemas

La dificultad teórica de un CSP recae enteramente en la topología de su **grafo de restricciones**,:

- **Componentes Conexas (Subproblemas independientes):** Si el grafo se puede fragmentar en zonas sin restricciones cruzadas, la complejidad se desploma. Si dividimos un problema de variables en subproblemas

independientes, la cantidad de asignaciones completas a procesar **cae de a** , permitiendo resolver en microsegundos problemas que de otro modo tardarían siglos,.

7. BÚSQUEDAS INSPIRADAS EN LA NATURALEZA

La naturaleza como fuente de inspiración

Consiste en **simular cómo la naturaleza resuelve problemas** mediante algoritmos inspirados en procesos químicos, leyes físicas o fenómenos biológicos evolutivos y sociales.

- Poseen un gran impacto y son altamente efectivos para resolver **grandes problemas de optimización**.
- Incorporan **mecánicas estocásticas (aleatorias)**, sacrificando la exploración sistemática de los árboles de búsqueda convencionales a cambio de eficiencia en paisajes masivos.
- Salvo el recocido simulado, la mayoría son **búsquedas locales basadas en poblaciones**, es decir, manejan múltiples estados simultáneos.

Recocido Simulado (Simulated Annealing - SA)

Inspirado en el **recocido**, un proceso metalúrgico donde un material se calienta a alta temperatura y luego se enfría progresivamente para mejorar sus propiedades estructurales (haciéndolo llegar a un estado cristalino de baja energía).

Es un **algoritmo de búsqueda local que utiliza un mecanismo probabilístico regulado por la "temperatura" para escapar de máximos o mínimos locales**.

- **Esquema de recocido o enfriamiento:** Comienza con una temperatura muy alta que **desciende gradualmente con el paso de las iteraciones** hasta llegar a un valor cercano a cero.
- **Mecanismo de escape:** En lugar de moverse siempre al mejor vecino (como Ascensión de Colinas), el algoritmo **evalúa a un sucesor aleatorio**. Si es mejor, lo acepta. **Si el sucesor empeora el estado, lo acepta con una cierta probabilidad**.
- **Función de probabilidad de aceptación ():** Matemáticamente, se utiliza la fórmula exponencial . De esto se desprenden dos reglas vitales:

1. Los "malos" movimientos son **más probables al comienzo (temperatura alta) y se vuelven muy improbables al final** de la ejecución.
 2. Fijada la temperatura, **la probabilidad de aceptar el sucesor empeora drásticamente mientras más perjudicial sea el movimiento.**
-

Algoritmo Genético (Genetic Algorithm - GA)

Inspirado en la teoría evolutiva de la **selección natural de Charles Darwin**. Simula la supervivencia del más apto a través de la **reproducción sexual** y mutaciones aleatorias.

Fases del algoritmo: **1. Población inicial:** Comienza con un conjunto de **individuos (estados) generados aleatoriamente**. El genoma de cada individuo se representa típicamente como una *string* (cadena) de números o bits sobre un alfabeto finito.

2. Función de fitness (Función objetivo): Cada individuo es evaluado para determinar su "aptitud". Para problemas de maximización, la función debe **devolver valores más altos para los mejores individuos**.

3. Selección: Consiste en **elegir aleatoriamente pares de individuos** de la generación actual para reproducirlos. Se utiliza una distribución no-uniforme donde **la probabilidad de que un individuo sea elegido es directamente proporcional a su valor de fitness**.

4. Cruzamiento (Crossover): Consiste en **generar nuevos individuos a partir de la mezcla de la información genética de los padres** seleccionados. Su ventaja teórica es la capacidad de combinar "bloques de genes" exitosos.

- *Problema en TSP:* En problemas de rutas como el del viajante, un cruce clásico puede romper la legalidad de la ruta (visitando ciudades dos veces o dejando sin visitar otras). Para solucionarlo, **se debe cambiar la representación del genoma (ej. Representación Ordinal)** u usar operadores de cruce parciales que reparen los duplicados.

5. Mutación: Se realiza un cambio aleatorio en el genoma del nuevo individuo con una **probabilidad muy baja**. Sirve para introducir y **mantener la diversidad genética**, evitando que la población se estanque en mínimos locales.

- **Evolución de las generaciones:** Inicialmente la población es muy diversa. A medida que avanzan las generaciones, **ocurre una convergencia y se pierde**

diversidad, ya que la selección natural se encarga de propagar las características de los mejores individuos en toda la población.

Optimización por Colonia de Hormigas (Ant Colony Optimization - ACO)

Inspirado en el comportamiento colectivo de los insectos y en cómo utilizan el **rastro de feromonas como medio de comunicación** para encontrar los caminos más cortos hacia la comida.

Fases del algoritmo: **1. Construcción de soluciones:** Una **población de hormigas** artificiales inicia en puntos aleatorios y **construye rutas de forma incremental y estocástica**.

- *Mecanismo:* En el problema del viajante (TSP), cada hormiga elige probabilísticamente qué ciudad visitar a continuación. **La probabilidad de elegir una arista es proporcional a la cantidad de feromona presente () e inversamente proporcional a la distancia real ()**. Es decir, es más probable elegir rutas cortas o muy transitadas por otras hormigas.

2. Depósito y evaporación: Antes de pasar a la siguiente iteración, se debe actualizar la cantidad global de feromonas del sistema mediante dos acciones conjuntas:

- **Depósito:** Se **incrementa el nivel de feromonas de las aristas recorridas** para reforzar componentes de construcción de buenas soluciones. Mientras más corto sea el tour final logrado por una hormiga, **más cantidad de feromona depositará** en su ruta.
 - **Evaporación:** Para evitar que todas las hormigas converjan prematuramente en el primer camino encontrado, **se reduce periódicamente un porcentaje () del nivel de feromona en todos los caminos**. Esto favorece el "olvido" y la exploración de nuevas rutas.
-

Optimización por Enjambre de Partículas (Particle Swarm Optimization - PSO)

Inspirado en el comportamiento coordinado de una **bandada de aves o un cardumen de peces**, donde una población en movimiento simula moverse como una entidad unificada pero cada miembro mantiene su trayectoria.

A diferencia de los demás, este algoritmo se utiliza para resolver problemas en espacios continuos, específicamente para **optimizar funciones reales no lineales ()**.

Componentes y funcionamiento: Un enjambre de partículas explora el espacio, moviéndose de manera fluida (actualizando vectores de velocidad y posición en cada iteración). Su movimiento está guiado de manera simultánea por dos tipos de memoria:

1. **Memoria Individual ():** Cada partícula guarda el registro de la **mejor posición personal** que ha visitado en toda su historia.
2. **Memoria Colectiva ():** El enjambre completo comparte información para identificar y sentirse atraído hacia la **mejor posición global** que haya encontrado cualquier partícula del conjunto.